

Floyd Warshal Algorithm

The Floyd–Warshall algorithm is a graph algorithm used to find the shortest paths between every pair of vertices in a weighted graph.

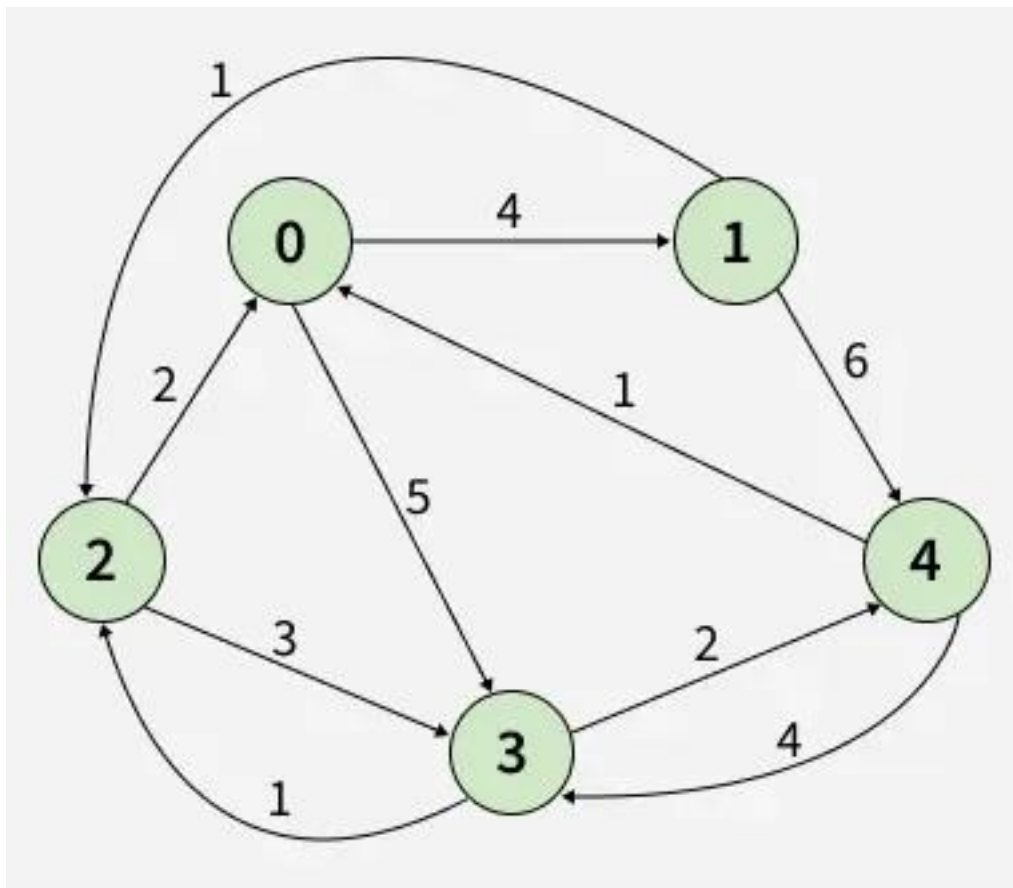
Basic idea: Suppose we want to find the shortest path from vertex $A \rightarrow C$. There may be two possibilities:

1. Direct path: $A \rightarrow C$
2. Path through another vertex: $A \rightarrow B \rightarrow C$

Floyd–Warshall checks whether going through an intermediate vertex gives a shorter path.

Floyd–Warshall is normally implemented using an adjacency matrix (distance matrix).

Example Graph:



<https://media.geeksforgeeks.org/wp-content/uploads/20251120110153925027/5.webp>

Taking Input of Adjacency Matrix:

```
#include<iostream>
#include<vector>
using namespace std;
const int inf = 1e8;
int main(){
    int v,e;
    cout<<"Enter number of vertex : ";
    cin>>v;
    cout<<"Enter number of edge : ";
    cin>>e;

    vector<vector<int>> adj(v,vector<int>(v));
    for(int i=0; i<v; i++){
        for(int j=0; j<v; j++){
            if(i==j){
                adj[i][j] = 0;
            }else{
                adj[i][j] = inf;
            }
        }
    }
    cout<<"Before taking edges input the adjacency matrix : \n";
    cout<<" ";
    for(int i=0; i<v; i++){
        cout<<" "<<i<<" ";
    }
    cout<<"\n";
    for(int i=0; i<v; i++){
        cout<<i;
        for(int j=0; j<v; j++){
            if(adj[i][j]==inf){
                cout<<"Inf ";
            }else{
                cout<<" "<<adj[i][j]<<" ";
            }
        }
        cout<<"\n";
    }
    cout<<"\n\nEnter Edges: \n";
    for(int i=0; i<e; i++){
        int u,v1,w;
        cin>>u>>v1>>w;
        adj[u][v1] = w;
    }
    cout<<"\n\n\n";
    cout<<"After taking edges input the adjacency matrix : \n";
```

```

cout<<" ";
for(int i=0; i<v; i++){
    cout<<" "<<i<<" ";
}
cout<<"\n";
for(int i=0; i<v; i++){
    cout<<i;
    for(int j=0; j<v; j++){
        if(adj[i][j]==inf){
            cout<<"Inf ";
        }else{
            cout<<" "<<adj[i][j]<<" ";
        }
    }
    cout<<"\n";
}

return 0;
}

```

Enter number of vertex : 5
 Enter number of edge : 10
 Before taking edges input the adjacency matrix :

	0	1	2	3	4
0	0	Inf	Inf	Inf	Inf
1	Inf	0	Inf	Inf	Inf
2	Inf	Inf	0	Inf	Inf
3	Inf	Inf	Inf	0	Inf
4	Inf	Inf	Inf	Inf	0

Enter Edges:

```

0 1 4
0 3 5
1 4 6
1 2 1
2 0 2
2 3 3
3 2 1
3 4 2
4 0 1
4 3 4

```

After taking edges input the adjacency matrix :

	0	1	2	3	4
0	0	4	Inf	5	Inf
1	Inf	0	1	Inf	6
2	2	Inf	0	3	Inf
3	Inf	Inf	1	0	2
4	1	Inf	Inf	4	0

Algorithm

1. Create a distance matrix D from the graph's adjacency matrix.
2. Set:
 - $D[i][i] = 0$
 - $D[i][j] = \text{edge weight}$ if an edge exists.
 - $D[i][j] = \infty$ if there is no direct edge.
3. For each vertex k , consider it as an intermediate vertex.
4. For every pair of vertices i and j , update:
$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$
5. After all vertices have been considered, $D[i][j]$ contains the shortest distance from vertex i to vertex j .

Implementation:

3. For each vertex k , consider it as an intermediate vertex.

```
for(int k=0; k<v; k++){  
  
}
```

4. For every pair of vertices i and j

```
for(int k=0; k<v; k++){  
    for(int i=0; i<v; i++){  
        for(int j=0; j<v; j++){  
  
        }  
    }  
}
```

update: $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$

```
for(int k=0; k<v; k++){  
    for(int i=0; i<v; i++){  
        for(int j=0; j<v; j++){  
            adj[i][j] = min(adj[i][j], adj[i][k]+adj[k][j]);  
        }  
    }  
}
```

Full Code:

```
#include<iostream>
#include<vector>
using namespace std;
const int inf = 1e8;
int main(){
    int v,e;
    cout<<"Enter number of vertex : ";
    cin>>v;
    cout<<"Enter number of edge : ";
    cin>>e;

    vector<vector<int>> adj(v,vector<int>(v));
    for(int i=0; i<v; i++){
        for(int j=0; j<v; j++){
            if(i==j){
                adj[i][j] = 0;
            }else{
                adj[i][j] = inf;
            }
        }
    }
    cout<<"Before taking edges input the adjacency matrix : \n";
    cout<<" ";
    for(int i=0; i<v; i++){
        cout<<" "<<i<<" ";
    }
    cout<<"\n";
    for(int i=0; i<v; i++){
        cout<<i;
        for(int j=0; j<v; j++){
            if(adj[i][j]==inf){
                cout<<"Inf ";
            }else{
                cout<<" "<<adj[i][j]<<" ";
            }
        }
        cout<<"\n";
    }
    cout<<"\n\nEnter Edges: \n";
    for(int i=0; i<e; i++){
        int u,v1,w;
        cin>>u>>v1>>w;
        adj[u][v1] = w;
    }
    cout<<"\n\n\n";
    cout<<"After taking edges input the adjacency matrix : \n";
```

```

cout<<" ";
for(int i=0; i<v; i++){
    cout<<" "<<i<<" ";
}
cout<<"\n";
for(int i=0; i<v; i++){
    cout<<i;
    for(int j=0; j<v; j++){
        if(adj[i][j]==inf){
            cout<<"Inf ";
        }else{
            cout<<" "<<adj[i][j]<<" ";
        }
    }
    cout<<"\n";
}

for(int k=0; k<v; k++){
    for(int i=0; i<v; i++){
        for(int j=0; j<v; j++){
            adj[i][j] = min(adj[i][j],adj[i][k]+adj[k][j]);
        }
    }
}

cout<<"\n\nAfter applying Floyd-warshall, the matrix : \n";
cout<<" ";
for(int i=0; i<v; i++){
    cout<<" "<<i<<" ";
}
cout<<"\n";
for(int i=0; i<v; i++){
    cout<<i;
    for(int j=0; j<v; j++){
        if(adj[i][j]==inf){
            cout<<"Inf ";
        }else{
            cout<<" "<<adj[i][j]<<" ";
        }
    }
    cout<<"\n";
}
return 0;
}

```

Enter number of vertex : 5

Enter number of edge : 10

Before taking edges input the adjacency matrix :

	0	1	2	3	4
0	0	Inf	Inf	Inf	Inf
1	Inf	0	Inf	Inf	Inf
2	Inf	Inf	0	Inf	Inf
3	Inf	Inf	Inf	0	Inf
4	Inf	Inf	Inf	Inf	0

Enter Edges:

```

0 1 4
0 3 5
1 4 6
1 2 1
2 0 2
2 3 3
3 2 1
3 4 2
4 0 1
4 3 4

```

After taking edges input the adjacency matrix :

	0	1	2	3	4
0	0	4	Inf	5	Inf
1	Inf	0	1	Inf	6
2	2	Inf	0	3	Inf
3	Inf	Inf	1	0	2
4	1	Inf	Inf	4	0

After applying Floyd-warshall, the matrix :

	0	1	2	3	4
0	0	4	5	5	7
1	3	0	1	4	6
2	2	6	0	3	5
3	3	7	1	0	2
4	1	5	5	4	0